ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

SmartParking

Ingénierie Informatique et Réseaux

Développement Digital & Systèmes d'Information

Cahier des charges de Projet :

Conception et mise en œuvre d'un Système Intelligent de Gestion des Parkings

Smart Parking System

Réalisé par :

AIT ICHOU Rafik
ABOD Brahim
BENASSOU Nousaire
MOUSSAOUI Moad

Encadré par :

Pr. ESSABBAR Driss

Année Universitaire : 2025–2026

Page des Signatures

Équipe du projet :

AIT ICHOU Rafik
Signature : _____

ABOD Brahim
Signature : _____

BENASSOU Nousaire
Signature : _____

MOUSSAOUI Moad
Signature : _____

Encadrant :

Pr. ESSABBAR Driss
Signature : _____

Fait à Marrakech , le 6 mars 2026

Liste des abréviations

API	Application Programming Interface
IA	Intelligence Artificielle
IoT	Internet of Things
JWT	JSON Web Token
HTTPS	HyperText Transfer Protocol Secure
TLS	Transport Layer Security
SQL	Structured Query Language
UML	Unified Modeling Language
UI	User Interface
UX	User Experience
REST	Representational State Transfer
ESP32-CAM	Microcontrôleur avec caméra intégrée pour applications IoT
YOLOv8	You Only Look Once version 8 (modèle de détection d'objets)
BCrypt	Algorithme de hachage sécurisé pour mots de passe
CRUD	Create, Read, Update, Delete
HTML	HyperText Markup Language

Table des figures

1 Logo de l'application Smart Parking System 1
2 Schéma détaillé du système de gestion de parking intelligent 5
3 Panorama global du Smart Parking System 7
4 Architecture globale du Smart Parking System 8
5 Scénarios de fonctionnement du Smart Parking System 11

Liste des tableaux

1 Relation entre les acteurs et les fonctionnalités du Smart Parking System 3

Table des matières

Page des Signatures i

Liste des abréviations ii

Table des figures iii

Liste des tableaux iv

Cahier des charges 1

1 Introduction . . . 1
- 1.1 Contexte et présentation du projet . . . 1
- 1.2 Problématique . . . 1
- 1.3 Objectifs du projet . . . 2
- 1.4 Portée du projet . . . 2

2 Parties prenantes et acteurs . . . 2
- 2.1 Parties prenantes . . . 2
- 2.2 Acteurs . . . 2

3 Besoins fonctionnels . . . 3
- 3.1 Description des fonctionnalités . . . 3
- 3.2 Acteurs et fonctionnalités . . . 3
- 3.3 Fonctionnement du système de détection automatique . . . 3
- 3.4 Fonctionnalités destinées aux utilisateurs . . . 4
- 3.5 Fonctionnalités de gestion (Administrateur) . . . 4
- 3.6 Fonctionnalités du système de paiement . . . 4
- 3.7 Fonctionnalités de supervision globale (Super Administrateur) . . . 5
- 3.8 Exigences Fonctionnelles Détaillées . . . 5

4 Besoins non fonctionnels . . . 6
- 4.1 Performance . . . 6
- 4.2 Sécurité . . . 6
- 4.3 Accessibilité et compatibilité . . . 6
- 4.4 Maintenance et évolutivité . . . 6

5 Architecture technique . . . 6
- 5.1 Vue d'ensemble du système . . . 6
- 5.2 Architecture logique . . . 8
  - 5.2.1 Couche Présentation (Frontend) . . . 8
  - 5.2.2 Couche Métier (Backend) . . . 8
  - 5.2.3 Couche Données (Base de données) . . . 9
  - 5.2.4 Couche IoT . . . 9
  - 5.2.5 Couche Intelligence Artificielle . . . 9
- 5.3 Flux de données du système . . . 10
- 5.4 Technologies et outils . . . 10
- 5.5 Scénarios de fonctionnement du système . . . 10

5.6 Contraintes techniques . . . 11
6 Planification du projet . . . 12
6.1 Phases du projet . . . 12
6.2 Planning du projet . . . 12
6.3 Pénalité en cas de retard . . . 13
7 Tests et validation . . . 13
7.1 Stratégie de tests . . . 13
7.2 Scénarios de tests . . . 13
7.3 Critères d'acceptation . . . 14
7.4 Livraison, documentation et modalités . . . 14
7.4.1 Livrables Techniques . . . 15
7.4.2 Documentation fonctionnelle et technique . . . 15
7.4.3 Product Backlog . . . 15
7.4.4 Scénarios de test et Plan de test . . . 15
7.4.5 Documentation utilisateur . . . 16
7.4.6 Soutenance et démonstration . . . 16
8 Analyse des Risques . . . 16
8.1 Risques techniques . . . 16
8.2 Risques liés à la cybersécurité . . . 16
8.3 Risques opérationnels et organisationnels . . . 17
8.4 Risques légaux et réglementaires . . . 17
9 Contraintes Légales et Réglementaires . . . 17
10 Conclusion . . . 17

Cahier des charges

1 Introduction

1.1 Contexte et présentation du projet

Figure 1 : Logo de l'application Smart Parking System

Dans les grandes agglomérations urbaines, la gestion des espaces de stationnement représente un défi majeur. L'augmentation continue du nombre de véhicules entraîne une saturation des parkings publics et privés, provoquant une perte de temps significative pour les conducteurs, une congestion accrue du trafic routier ainsi qu'une augmentation des émissions de gaz à effet de serre due à la recherche prolongée de places disponibles. Les systèmes traditionnels de gestion des parkings reposent généralement sur une surveillance humaine ou sur des dispositifs peu automatisés, incapables d'assurer une supervision en temps réel et une exploitation optimale des infrastructures. Cette absence d'automatisation limite la capacité des gestionnaires à analyser les données d'occupation et à optimiser la tarification et les revenus. Face à ces limites, le présent projet propose la conception et la mise en œuvre d'un Système Intelligent de Gestion des Parkings (Smart Parking System) basé sur l'intégration des technologies modernes telles que l'Internet des Objets (IoT), la vision par ordinateur et l'analyse de données en temps réel. Ce système vise à automatiser la détection des places disponibles, centraliser la gestion multi-parkings et améliorer l'efficacité opérationnelle des infrastructures de stationnement.

1.2 Problématique

La problématique principale du projet réside dans l'absence d'un système intelligent capable d'assurer une détection automatique fiable des places de stationnement et une gestion centralisée en temps réel des infrastructures de parking. Cette lacune entraîne une sous-exploitation des espaces disponibles, une mauvaise gestion des flux de véhicules et une absence d'analyse statistique avancée permettant d'optimiser les décisions stratégiques. Il devient donc nécessaire de concevoir une solution technologique innovante capable de répondre simultanément aux exigences de performance, de

fiabilité, de sécurité et d'évolutivité.

1.3 Objectifs du projet

L'objectif général du projet est de développer une plateforme centralisée permettant la gestion intelligente de plusieurs parkings à travers un système unique. Les objectifs spécifiques et mesurables sont les suivants :

- Mettre en place un système de détection automatique des places avec une précision minimale de 95
- Garantir un temps de réponse inférieur à 2 secondes pour toute requête utilisateur.
- Permettre la gestion simultanée d'au moins 500 connexions utilisateurs.
- Implémenter un système de paiement électronique sécurisé assurant la traçabilité complète des transactions.
- Fournir un tableau de bord interactif permettant l'analyse des taux d'occupation et des revenus générés.
- Concevoir une architecture modulaire facilitant l'ajout futur de nouveaux parkings sans modification majeure du système existant.

1.4 Portée du projet

Le projet inclura le développement d'une application web permettant la gestion centralisée des parkings, l'intégration de dispositifs IoT pour la collecte des données en temps réel, ainsi qu'un module d'intelligence artificielle destiné à la détection automatique des places disponibles. Il comprendra également la gestion des utilisateurs, la configuration des tarifs, le traitement des paiements électroniques et la génération de rapports statistiques. En revanche, certaines fonctionnalités ne seront pas incluses dans la première version du système, notamment l'intégration avec des systèmes gouvernementaux externes, le développement d'une application mobile native complète et l'implémentation d'un système avancé de reconnaissance faciale.

2 Parties prenantes et acteurs

2.1 Parties prenantes

Les principales parties prenantes du projet sont les gestionnaires de parkings, les administrateurs du système, les conducteurs utilisateurs du service ainsi que l'équipe de développement chargée de la conception et de la maintenance de la solution.

Le système devra répondre aux besoins opérationnels des gestionnaires tout en offrant une expérience fluide et intuitive aux utilisateurs finaux.

2.2 Acteurs

Le système distinguera plusieurs acteurs principaux :

- **Le Super Administrateur** : sera responsable de la gestion globale de l'application, notamment la création et la supervision des administrateurs, la gestion des parkings ainsi que la consultation des statistiques globales du système.
- **L'Administrateur de parking** : sera chargé de gérer un parking spécifique, de configurer les tarifs, de consulter l'état des places en temps réel et de superviser les transactions associées à son parking.
- **L'Utilisateur (conducteur)** : pourra consulter les places disponibles, réserver une place de stationnement, gérer ses réservations et effectuer des paiements en ligne sécurisés.

- **Le Système de Paiement** : constitue un acteur externe chargé de valider les transactions financières, de sécuriser les paiements électroniques et de retourner le statut de la transaction au système.

3 Besoins fonctionnels

3.1 Description des fonctionnalités

Le système reposera sur une gestion intelligente et centralisée des parkings. Il permettra la création et la configuration de plusieurs parkings avec leurs caractéristiques respectives telles que la localisation, la capacité et la tarification.
Un module de vision par ordinateur analysera les images capturées par les caméras afin de détecter automatiquement les places libres et occupées.
Ces informations seront transmises en temps réel au serveur central et affichées sur le tableau de bord. Le système offrira également aux utilisateurs la possibilité de réserver une place et d'effectuer un paiement électronique sécurisé.
Les administrateurs pourront consulter des rapports détaillés concernant le taux d'occupation, les revenus générés et les statistiques d'utilisation.
Une gestion des rôles et des permissions sera mise en place afin de garantir un contrôle d'accès adapté à chaque profil d'utilisateur.

3.2 Acteurs et fonctionnalités

Les interactions entre les différents acteurs du système et les fonctionnalités principales peuvent être résumées comme suit :

- Le **Super Administrateur** pourra gérer les administrateurs et superviser l'ensemble des parkings.
- L'**Administrateur de parking** pourra configurer les paramètres de son parking et consulter les statistiques associées.
- L'**Utilisateur** pourra consulter les places disponibles, effectuer une réservation et réaliser un paiement en ligne.

Afin de représenter de manière plus structurée les relations entre les acteurs et les fonctionnalités du système, la matrice suivante est proposée :

Fonctionnalités	**Super Admin**	**Admin Parking**	**Utilisateur**
Gestion des administrateurs	X		
Gestion des parkings	X		
Configuration des tarifs		X	
Consultation des statistiques	X	X	
Consultation des places disponibles		X	X
Réservation d'une place			X
Paiement en ligne			X
Supervision globale du système	X		

Table 1 : Relation entre les acteurs et les fonctionnalités du Smart Parking System

3.3 Fonctionnement du système de détection automatique

Lorsqu'un véhicule entre dans le parking, une caméra positionnée au niveau de la zone surveillée capture automatiquement une image.

Cette image est ensuite traitée par un module d'intelligence artificielle chargé d'analyser l'état des places de stationnement.

Après traitement, chaque place est classifiée comme **libre** ou **occupée**.

Les résultats sont ensuite :

- Transmis au serveur central
- Enregistrés dans la base de données
- Affichés en temps réel sur le tableau de bord de supervision

Ce mécanisme permet une mise à jour automatique et continue de l'état du parking.

3.4 Fonctionnalités destinées aux utilisateurs

Le système met à disposition des utilisateurs une interface web/mobile permettant :

- La consultation des places disponibles en temps réel
- La sélection d'une place de stationnement
- La réservation d'une place
- Le paiement en ligne sécurisé
- La réception d'une confirmation de réservation sous forme de notification

L'objectif est d'offrir une expérience simple, rapide et intuitive.

3.5 Fonctionnalités de gestion (Administrateur)

L'administrateur dispose d'un espace de gestion lui permettant :

- D'ajouter, modifier ou supprimer un parking
- De configurer les tarifs de stationnement
- De consulter les rapports d'activité
- De superviser l'état des places en temps réel

Ces fonctionnalités garantissent une gestion efficace des infrastructures.

3.6 Fonctionnalités du système de paiement

Le Smart Parking System intégrera un module de paiement électronique sécurisé permettant aux utilisateurs de régler leurs réservations ou leur stationnement après utilisation.

Le module de paiement devra garantir les fonctionnalités suivantes :

- Permettre le paiement via plusieurs moyens sécurisés tels que les cartes bancaires et les portefeuilles électroniques.
- Intégrer des API de paiement externes sécurisées et conformes aux normes internationales de sécurité des transactions électroniques (PCI-DSS).
- Vérifier la validité des transactions côté serveur avant toute confirmation de paiement.
- Chiffrer les données sensibles échangées entre le client et le serveur via des protocoles sécurisés (HTTPS/TLS).
- Générer automatiquement un reçu électronique pour chaque transaction validée.
- Enregistrer l'ensemble des transactions financières dans la base de données pour assurer la traçabilité et l'audit des opérations.
- Gérer les cas d'annulation et de remboursement selon les règles définies par l'administrateur du système.
- Envoyer une notification instantanée à l'utilisateur après chaque opération de paiement.

3.7 Fonctionnalités de supervision globale (Super Administrateur)

Le Super Administrateur assure la gestion globale du système. Il peut :

- Créer, modifier ou supprimer des comptes administrateurs
- Ajouter de nouveaux parkings au système
- Consulter des statistiques globales couvrant l'ensemble des parkings
- Superviser l'activité générale de la plateforme

Ce niveau d'accès permet une administration centralisée et stratégique.

3.8 Exigences Fonctionnelles Détaillées

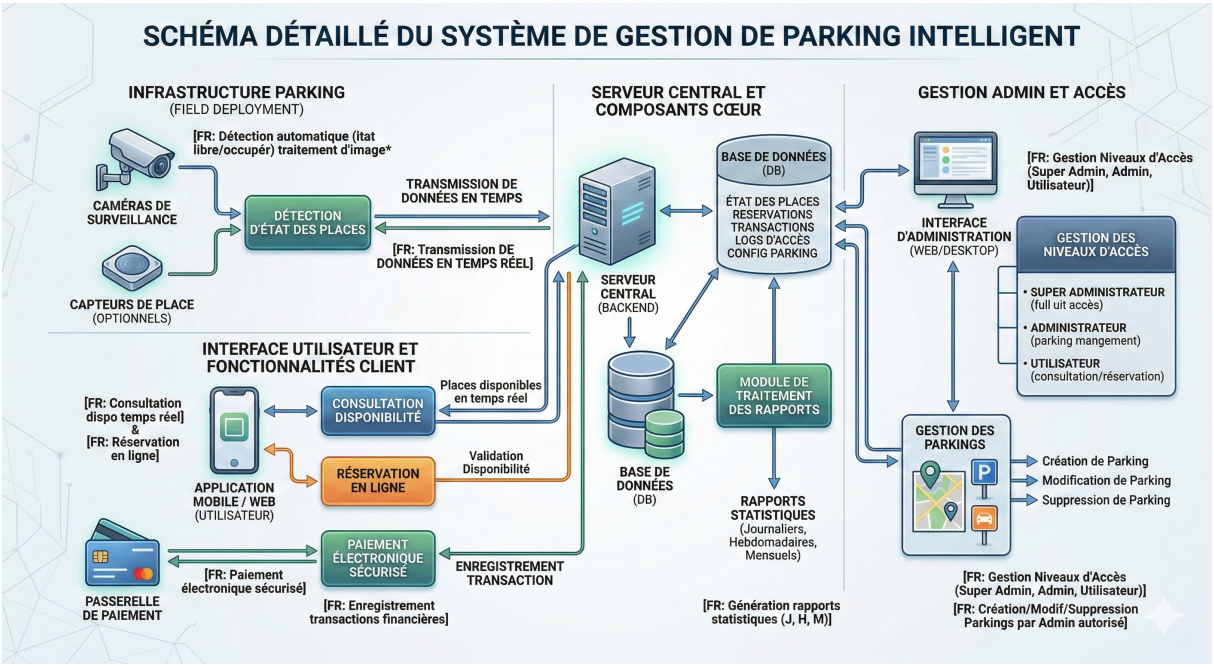

Figure 2 : Schéma détaillé du système de gestion de parking intelligent

Afin de formaliser les besoins fonctionnels, les exigences suivantes sont définies :

- Le système doit permettre la création, la modification et la suppression de parkings par un administrateur autorisé.
- Le système doit détecter automatiquement l'état (libre/occupée) de chaque place de stationnement à partir des images capturées par les caméras connectées.
- Le système doit transmettre les données de détection en temps réel vers le serveur central.
- Le système doit permettre aux utilisateurs de consulter les places disponibles en temps réel.
- Le système doit permettre la réservation en ligne d'une place disponible.
- Le système doit intégrer un module de paiement électronique sécurisé.
- Le système doit enregistrer toutes les transactions financières dans la base de données.
- Le système doit générer des rapports statistiques journaliers, hebdomadaires et mensuels.
- Le système doit gérer différents niveaux d'accès (Super Administrateur, Administrateur, Utilisateur).

4 Besoins non fonctionnels

4.1 Performance

Le système devra garantir un temps de réponse inférieur à deux secondes pour chaque action utilisateur. La plateforme devra supporter un minimum de 500 connexions simultanées sans dégradation notable des performances. La disponibilité du système devra être supérieure à 99 % sur une période annuelle. La mise à jour des données issues des dispositifs IoT devra s'effectuer en temps réel avec une latence inférieure à 1 seconde.

4.2 Sécurité

L'authentification des utilisateurs sera basée sur un mécanisme de jetons JWT avec une durée d'expiration configurable. Les mots de passe seront stockés sous forme chiffrée en utilisant l'algorithme BCrypt. Toutes les communications entre le client et le serveur seront sécurisées via le protocole HTTPS utilisant TLS 1.3. Les transactions financières devront respecter les standards de sécurité des paiements électroniques. Un système de gestion des rôles et permissions garantira un contrôle d'accès strict aux ressources. Le système respectera les principes de protection des données personnelles conformément aux réglementations en vigueur. Les données sensibles seront chiffrées et l'accès sera strictement limité aux utilisateurs autorisés.

Sécurité des transactions financières
Pour garantir la sécurité des paiements en ligne :

- Les informations sensibles (numéro de carte, CVV) ne doivent jamais être stockées en clair.
- Toutes les communications avec la passerelle de paiement doivent être chiffrées (HTTPS/TLS).
- Le système doit détecter et prévenir les fraudes potentielles.
- Les transactions doivent être horodatées et enregistrées pour audit.

4.3 Accessibilité et compatibilité

L'application devra être compatible avec les navigateurs modernes tels que :

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

L'interface utilisateur devra être responsive et adaptée aux ordinateurs, tablettes et smartphones.

4.4 Maintenance et évolutivité

L'architecture du système sera modulaire et distribuée afin de permettre l'ajout futur de nouveaux parkings ou de nouvelles fonctionnalités. Des mises à jour régulières pourront être effectuées sans interruption majeure du service.

5 Architecture technique

5.1 Vue d'ensemble du système

La figure 3 présente une vue panoramique globale du Smart Parking System. Elle illustre l'ensemble des composants matériels et logiciels ainsi que les flux de données qui circulent entre les différentes couches du système.

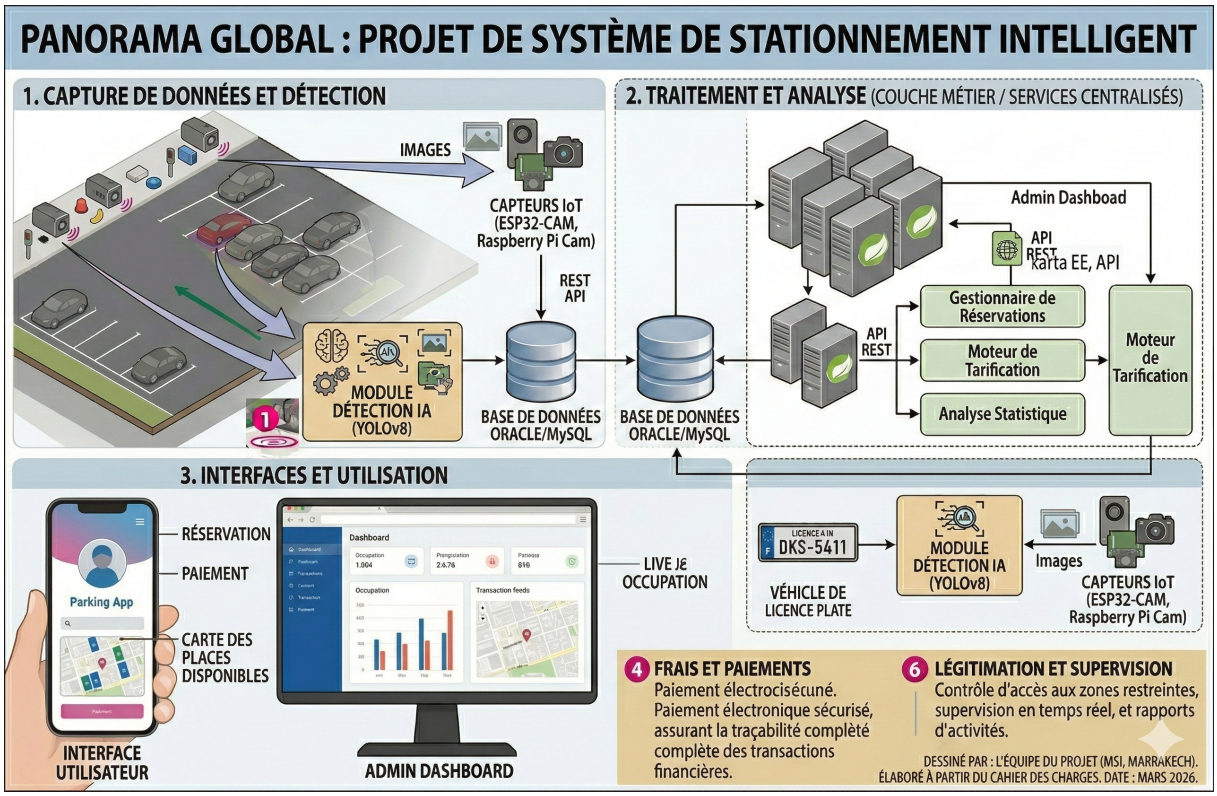

Figure 3 : Panorama global du Smart Parking System

Comme illustré dans la figure, l'architecture du système est organisée en trois grandes couches fonctionnelles.

1. Couche de capture et détection : Cette couche regroupe les capteurs IoT (ESP32-CAM, Raspberry Pi Cam) chargés de capturer en temps réel les images des zones de stationnement. Les flux d'images sont transmis au module d'intelligence artificielle basé sur le modèle YOLOv8, responsable de la classification des places en *libres* ou *occupées*. Les résultats sont ensuite envoyés vers le serveur central via une API REST sécurisée.

2. Couche de traitement et d'analyse (couche métier) : Elle constitue le cœur du système. Elle comprend le backend applicatif (Spring Boot ou Jakarta EE), la base de données relationnelle (Oracle/MySQL), ainsi que les services métiers tels que :

- le gestionnaire de réservations,
- le moteur de tarification,
- le module d'analyse statistique.

Cette couche assure la centralisation des données, la gestion des utilisateurs, le traitement des transactions financières et la mise à jour en temps réel de l'état des parkings.

3. Couche d'interfaces et d'utilisation : Elle comprend l'application web/mobile destinée aux utilisateurs pour la consultation des places, la réservation et le paiement électronique sécurisé. Elle inclut également le tableau de bord administrateur permettant la supervision en temps réel, l'analyse des indicateurs de performance et la génération de rapports.

Enfin, le système peut intégrer un module de reconnaissance de plaques d'immatriculation afin d'automatiser le contrôle d'accès et d'améliorer la sécurité du parking.

Cette organisation modulaire garantit la séparation des responsabilités, facilite la maintenance du système et assure son évolutivité dans un contexte multi-parkings.

5.2 Architecture logique

La figure 4 illustre l'architecture logique du Smart Parking System. Le système est structuré selon une approche multi-couches afin d'assurer la modularité, la maintenabilité et la séparation des responsabilités.

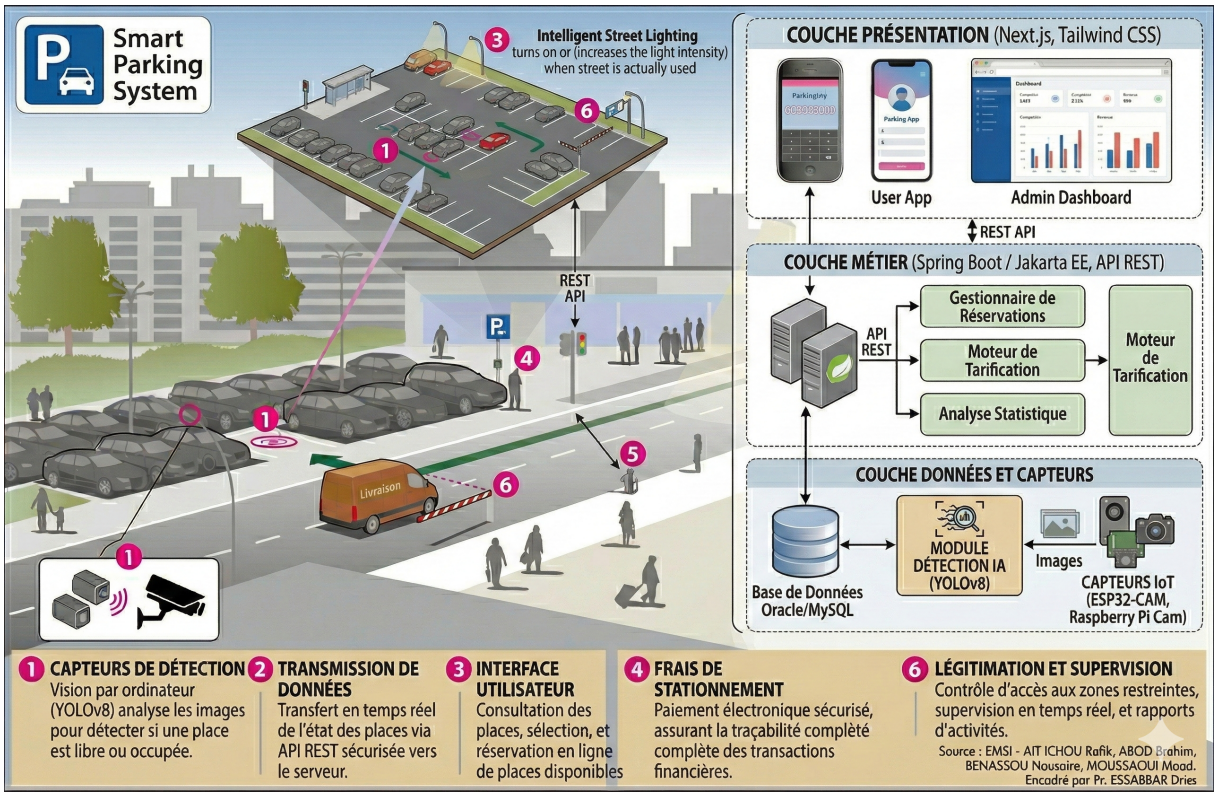

Figure 4 : Architecture globale du Smart Parking System

L'architecture est organisée en cinq couches principales :

5.2.1 Couche Présentation (Frontend)

Cette couche représente l'interface utilisateur du système. Elle peut être développée sous forme d'application web ou mobile. Elle permet aux utilisateurs de :

- Consulter la disponibilité des places en temps réel,
- Effectuer une réservation,
- Procéder au paiement électronique sécurisé,
- Consulter l'historique de leurs transactions.

Elle communique avec le backend via des API REST sécurisées (HTTP/HTTPS) et assure l'affichage dynamique des données.

5.2.2 Couche Métier (Backend)

Il s'agit du cœur du système. Cette couche implémente la logique applicative et gère les règles métier telles que :

- La gestion des utilisateurs et des rôles,
- Le traitement des réservations,
- Le calcul des tarifs,

- La gestion des paiements,
- La synchronisation des données IoT.

Le backend assure également la sécurité (authentification, autorisation) et l'intégrité des transactions. Il agit comme intermédiaire entre la couche présentation, la couche données et les modules IoT/IA.

5.2.3 Couche Données (Base de données)

Cette couche stocke l'ensemble des informations du système :

- Données des utilisateurs,
- Informations sur les parkings et les places,
- Historique des réservations,
- Transactions financières,
- Journaux d'activité.

Une base de données relationnelle (MySQL ou Oracle) est utilisée afin d'assurer la cohérence, la persistance et la sécurité des données.

5.2.4 Couche IoT

Cette couche comprend les dispositifs matériels installés sur le parking, notamment :

- Caméras ESP32-CAM,
- Capteurs de détection,
- Modules de communication réseau.

Elle assure la collecte des données en temps réel (images ou états des places) et leur transmission vers le serveur via des protocoles de communication adaptés (Wi-Fi, HTTP, MQTT).

5.2.5 Couche Intelligence Artificielle

Cette couche intègre le modèle de vision par ordinateur (YOLOv8) chargé de :

- Détecter les véhicules,
- Identifier les places occupées ou libres,
- Améliorer la précision du système.

Le module IA traite les images issues des caméras IoT et renvoie les résultats au backend pour mise à jour en temps réel de la base de données.

Cette architecture en couches garantit :

- Une séparation claire des responsabilités,
- Une meilleure évolutivité du système,
- Une maintenance simplifiée,
- Une possibilité d'intégration future de nouveaux services (analyse prédictive, reconnaissance de plaques, etc.).

5.3 Flux de données du système

Le fonctionnement global du **Smart Parking System** repose sur un échange structuré de données entre les composants matériels et logiciels. Le processus suit les étapes suivantes :

1. La caméra **ESP32-CAM / Raspberry Pi Cam** capture une image de la zone de stationnement en temps réel.
2. L'image est transmise au module d'intelligence artificielle hébergé sur un **Raspberry Pi**.
3. Le modèle de détection basé sur **YOLOv8** analyse l'image et détermine l'état de chaque place (libre ou occupée).
4. Les résultats du traitement sont envoyés via une API REST sécurisée vers le serveur Backend.
5. Le serveur met à jour la base de données en enregistrant l'état actuel des places.
6. Les informations mises à jour sont affichées en temps réel sur le tableau de bord administrateur et sur l'interface utilisateur.

Ce mécanisme garantit une synchronisation continue entre le module de détection et la plateforme centrale.

5.4 Technologies et outils

- **Backend** : Spring Boot ou Jakarta EE.
- **Frontend** : Next.js, HTML, CSS et Tailwind CSS.
- **Base de données** : Oracle Database/MySQL.
- **Vision par ordinateur** : OpenCV (Python) et YOLOv8.
- **Matériel** : ESP32-CAM, Raspberry Pi, caméra, Arduino.
- **Paiement en ligne** : Stripe, PayPal ou autre API sécurisée.

5.5 Scénarios de fonctionnement du système

La figure suivante illustre les différents scénarios de fonctionnement du Smart Parking System. Elle montre les interactions entre les utilisateurs, les dispositifs IoT, le module d'intelligence artificielle et le serveur backend.

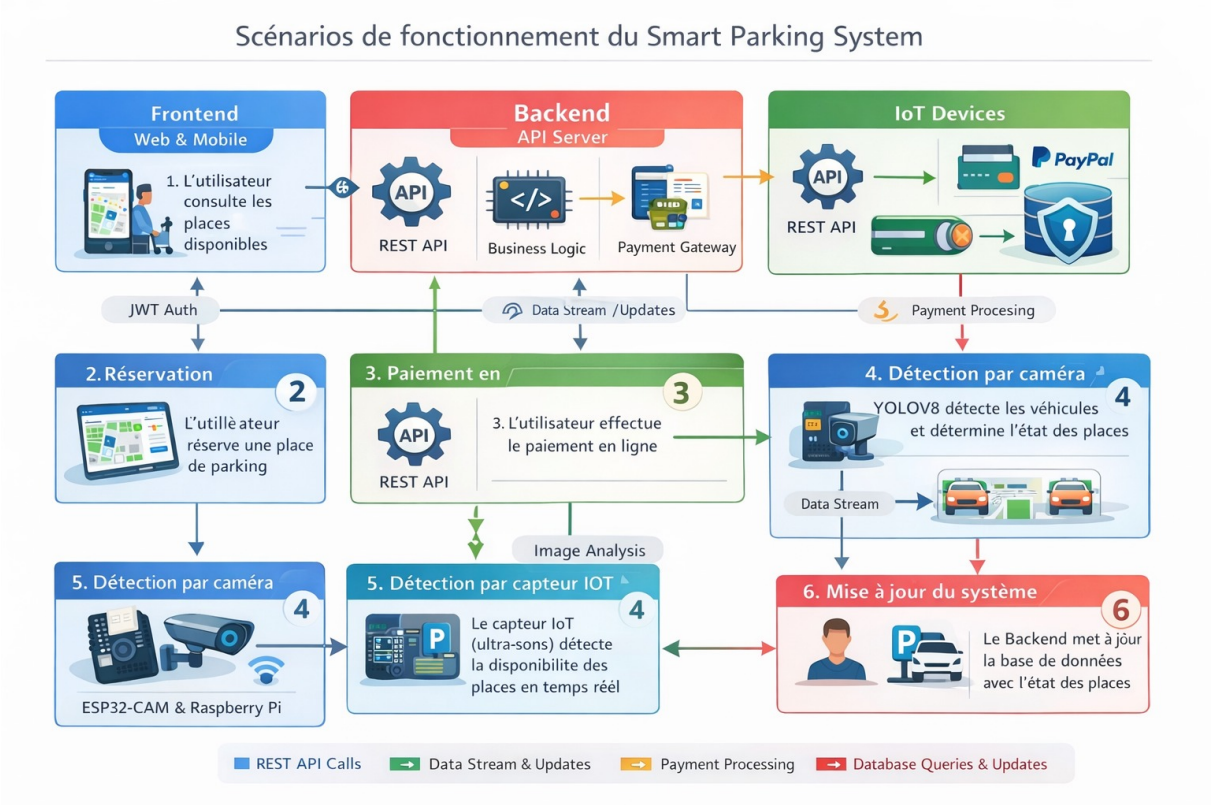

Figure 5 : Scénarios de fonctionnement du Smart Parking System

Ce schéma met en évidence les principales étapes du système :

- Consultation des places disponibles par l'utilisateur.
- Réservation d'une place de stationnement.
- Traitement du paiement électronique.
- Capture et analyse des images par les caméras IoT.
- Mise à jour en temps réel de l'état des places dans la base de données.
- Affichage des informations sur le tableau de bord administrateur.

5.6 Contraintes techniques

Le développement et le déploiement du **Smart Parking System** impliquent plusieurs contraintes techniques essentielles afin d'assurer la fiabilité, la performance et la sécurité du système :

- **Connexion Internet stable :** La plateforme repose sur la transmission en temps réel des données IoT et des images issues des caméras de surveillance. Une connexion Internet stable et performante est indispensable pour garantir que les informations de disponibilité des places et les notifications aux utilisateurs soient actualisées sans latence notable.
- **Caméras haute résolution et éclairage adéquat :** Les caméras doivent offrir une résolution suffisante pour permettre au module de vision par ordinateur de détecter avec précision l'état des places de stationnement. Des conditions d'éclairage constantes ou des systèmes d'éclairage additionnels sont nécessaires pour éviter les erreurs de détection liées à l'obscurité ou aux variations de luminosité.
- **Infrastructure serveur performante :** Le serveur central doit pouvoir gérer simultanément le traitement des flux d'images, les requêtes utilisateurs, les transactions financières et la mise à jour de la base de données. Il est recommandé d'utiliser des serveurs avec une puissance

de calcul adaptée (CPU multi-cœurs, mémoire vive suffisante et éventuellement GPU pour le traitement IA) afin de garantir un temps de réponse inférieur à 2 secondes pour toutes les opérations.

- **Disponibilité et redondance :** Afin de limiter les interruptions de service, des mécanismes de redondance et de sauvegarde doivent être prévus. Cela inclut notamment des serveurs de secours, des sauvegardes régulières de la base de données et des systèmes de cache local pour les modules IoT en cas de panne réseau temporaire.
- **Compatibilité matérielle et logicielle :** Les modules IoT (ESP32-CAM, Raspberry Pi, capteurs divers) doivent être compatibles avec les protocoles de communication choisis (HTTP, MQTT, Wi-Fi) et avec le backend. De même, les frameworks et bibliothèques utilisés pour le traitement d'images et l'IA doivent être compatibles avec le système d'exploitation du serveur et les composants matériels.
- **Sécurité technique :** La transmission des données et des images doit être sécurisée via des protocoles cryptés (HTTPS/TLS 1.3). Les serveurs doivent être configurés pour prévenir les intrusions, les accès non autorisés et les attaques par déni de service.
- **Intégration du module de paiement :** Le système de paiement doit être sécurisé et conforme aux normes PCI-DSS. Il doit pouvoir traiter simultanément plusieurs transactions sans ralentir la plateforme. Les API utilisées doivent être fiables, supportées et maintenues, et toutes les communications avec le serveur de paiement doivent être cryptées (HTTPS/TLS 1.3).

Ces contraintes techniques constituent le socle nécessaire pour assurer la performance, la fiabilité et la pérennité du Smart Parking System, tout en garantissant une expérience utilisateur fluide et sécurisée.

6 Planification du projet

6.1 Phases du projet

Le projet sera réalisé sur une durée estimée de 3 mois. Il suivra une approche structurée comprenant l'analyse des besoins, la conception technique, le développement, l'intégration des modules IoT et IA, les tests, puis le déploiement final.

6.2 Planning du projet

Durée estimée : 3 mois.
Le projet sera réparti comme suit :
Mois 1 :

- Analyse des besoins
- Rédaction du cahier des charges
- Conception UML
- Choix des technologies

Mois 2 :

- Développement Backend
- Conception base de données
- Implémentation API REST

Mois 3 :

- Développement Frontend
- Intégration module IA

- Intégration IoT
- Tests unitaires et d'intégration
- Optimisation performance
- Déploiement et documentation finale

6.3 Pénalité en cas de retard

Afin de garantir le respect du planning prévisionnel et l'atteinte des objectifs fixés, une clause de pénalité en cas de retard est définie pour le projet Smart Parking System. Tout retard dans la livraison des livrables techniques, fonctionnels ou dans la mise en production finale pourra entraîner des conséquences financières proportionnelles à l'impact du retard.

- **Retard dans les livrables intermédiaires :** Chaque livrable non livré à la date prévue pourra générer une pénalité correspondant à 0,5% du budget alloué au module concerné par jour de retard.
- **Retard dans la livraison de la version finale :** Le non-respect de la date de mise en production finale pourra engendrer une pénalité équivalente à 1% du budget total du projet par semaine de retard.
- **Plafond des pénalités :** Le montant total des pénalités appliquées ne pourra pas dépasser 10% du montant total du paiement prévu pour le projet.
- **Droit d'annulation du contrat :** Si le retard cumulé dépasse une durée de 30 jours calendaires par rapport au planning initial, le client se réserve le droit d'annuler le contrat du projet. Dans ce cas, une pénalité pouvant atteindre 10% du montant total du projet pourra être appliquée.

Cette clause vise à responsabiliser l'équipe projet, limiter les dérives du calendrier et assurer une planification stricte tout en restant équitable vis-à-vis des parties prenantes.

7 Tests et validation

7.1 Stratégie de tests

Le système sera soumis à des tests unitaires, des tests d'intégration et des tests de charge. Il sera considéré comme validé si la détection atteint un taux de précision supérieur à 95 %, si le temps de réponse respecte les spécifications et si les transactions sont enregistrées sans erreur.

7.2 Scénarios de tests

Afin de garantir la fiabilité et la performance du système, plusieurs scénarios de tests seront mis en place.

- **Scénario 1 : Détection automatique d'une place** Un véhicule se gare dans une place de stationnement. La caméra capture l'image de la zone concernée. Le module d'intelligence artificielle analyse l'image et classe la place comme « occupée ». L'information est ensuite transmise au serveur central, enregistrée dans la base de données et affichée sur le tableau de bord en temps réel.
- **Scénario 2 : Consultation des places disponibles** Un utilisateur accède à l'interface web afin de consulter les places libres. Le système interroge la base de données et affiche les informations actualisées en moins de deux secondes.
- **Scénario 3 : Réservation d'une place** L'utilisateur sélectionne une place disponible et confirme la réservation. Le système vérifie la disponibilité, enregistre la réservation et génère une confirmation.

- **Scénario 4 : Paiement électronique** Après la réservation ou à la sortie du parking, l'utilisateur effectue un paiement en ligne. Le système valide la transaction, met à jour l'état de la place et enregistre la transaction dans la base de données.
- **Scénario 5 : Consultation des statistiques** L'administrateur se connecte au tableau de bord pour consulter les taux d'occupation et les revenus générés. Les données doivent s'afficher correctement et refléter les informations en temps réel.
- **Scénario 6 : Paiement électronique sécurisé** Après avoir sélectionné une place et confirmé la réservation, l'utilisateur effectue le paiement en ligne. Le système valide la transaction, met à jour l'état de la place et enregistre toutes les informations nécessaires pour la traçabilité. Des tests seront effectués pour vérifier :
  - La réussite de la transaction
  - L'enregistrement correct dans la base de données
  - La notification envoyée à l'utilisateur

7.3 Critères d'acceptation

Le projet sera considéré comme valide si les conditions suivantes sont respectées :

- Le taux de précision du système de détection automatique des places est supérieur ou égal à 95 %.
- Toutes les transactions de paiement sont enregistrées sans erreur et correctement associées aux utilisateurs concernés.
- Les données issues des dispositifs IoT sont synchronisées en temps réel avec le serveur central.
- Le temps de réponse du système est inférieur à deux secondes pour chaque action utilisateur.
- Le système supporte un nombre important de connexions simultanées sans dégradation notable des performances.
- Les mécanismes de sécurité (authentification, autorisation, chiffrement) fonctionnent correctement.

Ces critères garantiront la fiabilité, la performance et la robustesse globale du système.

7.4 Livraison, documentation et modalités

Le projet sera livré en deux phases distinctes :
Phase 1 : Version Bêta Cette version permettra :

- La validation des fonctionnalités principales
- Les tests en environnement réel
- L'identification des anomalies
- La collecte des retours utilisateurs

Phase 2 : Version Finale Après correction des anomalies et optimisation des performances, la version finale intégrera :

- L'ensemble des fonctionnalités validées
- Les améliorations issues des tests
- Les optimisations de sécurité et de performance

7.4.1 Livrables Techniques

Les livrables techniques comprendront :

Architecture du système

- Description détaillée de l'architecture globale (front-end, back-end, base de données, module IA)
- Schéma d'architecture technique (architecture 3-tiers ou microservices selon le choix)
- Description des flux de données
- Diagramme de déploiement (Optionnel)
- Description de l'infrastructure (serveur, cloud, caméras, API) (Optionnel)

Templates et maquettes

- Maquettes UI/UX (interfaces utilisateur)
- Templates des pages principales :
  - Page d'accueil
  - Tableau de bord administrateur
  - Interface utilisateur
  - Page de réservation
- Charte graphique utilisée

7.4.2 Documentation fonctionnelle et technique

Une documentation complète sera fournie, comprenant :

- Les spécifications fonctionnelles détaillées
- Les diagrammes UML
- Les spécifications des API REST
- Le modèle de base de données
- Le guide d'installation et de déploiement
- Les contraintes techniques et exigences système

7.4.3 Product Backlog

Un Product Backlog sera fourni dans une approche de gestion de projet agile. Il comprendra :

- La liste priorisée des fonctionnalités
- Les User Stories
- Les critères d'acceptation
- La priorisation (Must Have, Should Have, Could Have)
- L'estimation des tâches (complexité ou story points)

Ce document permettra d'assurer le suivi et l'évolution du projet.

7.4.4 Scénarios de test et Plan de test

Scénarios de test

Les scénarios de test décriront les cas pratiques permettant de valider :

- La détection automatique des places
- La mise à jour en temps réel
- La réservation d'une place

- Le paiement en ligne
- La gestion administrative

Plan de test
Le plan de test précisera :

- La stratégie de test (tests unitaires, tests d'intégration, tests fonctionnels)
- Les environnements de test
- Les outils utilisés
- Les critères d'acceptation
- Les indicateurs de performance (temps de réponse < 2 secondes)

7.4.5 Documentation utilisateur

Un manuel d'utilisation sera fourni pour :

- Les utilisateurs finaux
- Les administrateurs
- Les super administrateurs

Il expliquera :

- La consultation des places
- La gestion des réservations
- L'administration des parkings
- L'analyse des statistiques

7.4.6 Soutenance et démonstration

Une démonstration fonctionnelle complète du système sera réalisée lors de la soutenance afin de présenter :

- Le fonctionnement du module de détection
- Le processus de réservation
- Le tableau de bord administrateur
- Les statistiques globales

8 Analyse des Risques

Dans le cadre du développement du **Smart Parking System**, plusieurs risques ont été identifiés.

8.1 Risques techniques

- Précision du module de vision par ordinateur : erreurs possibles dues à une mauvaise qualité des images, un éclairage insuffisant ou un angle de capture inadapté.
- Défaillance du réseau Internet : interruption ou lenteur pouvant empêcher la transmission en temps réel des données IoT vers le serveur central.

8.2 Risques liés à la cybersécurité

- Accès non autorisés et attaques informatiques : risque de fuite de données sensibles ou altération des transactions financières.
- Tentatives de déni de service ou intrusion malveillante sur le serveur.

8.3 Risques opérationnels et organisationnels

- Retard dans le développement ou l'intégration des modules IoT et IA, pouvant impacter le planning global du projet.
- Difficultés de maintenance ou d'évolution du système lors de l'ajout de nouveaux parkings ou fonctionnalités.
- Risques liés à la coordination de l'équipe projet et aux dépendances entre les différents modules.

8.4 Risques légaux et réglementaires

- Non-conformité avec les réglementations sur la protection des données personnelles.
- Utilisation inappropriée ou conservation excessive des images capturées par les caméras.
- Non-respect des normes de sécurité pour les transactions financières électroniques.

9 Contraintes Légales et Réglementaires

Le système devra respecter les réglementations relatives à la protection des données personnelles. Les informations collectées concernant les utilisateurs seront traitées conformément aux principes de confidentialité et de minimisation des données.
Les images capturées par les caméras devront être utilisées uniquement à des fins de détection des places et ne devront pas être conservées au-delà de la durée nécessaire au traitement.
Les transactions financières devront être conformes aux normes de sécurité des paiements électroniques en vigueur.

10 Conclusion

Le présent cahier des charges définit l'ensemble des exigences fonctionnelles et non fonctionnelles du Smart Parking System. Il constitue le document de référence pour la conception, le développement, l'intégration et la validation du projet.
L'approche adoptée repose sur une architecture modulaire intégrant des technologies modernes telles que l'IoT, l'intelligence artificielle et les architectures web distribuées.
Le respect des performances, de la sécurité et de l'évolutivité constitue un élément central du projet. Ce document servira de base contractuelle et technique pour l'ensemble des phases du développement jusqu'à la soutenance finale.